Mobile Demo Script Explanation -- Goingnative (Reversing, Challenge 03)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: The native library's exported JNI function can be disassembled directly to recover the exact input string and constant it requires.

Background -- what the terms below mean

What does "Reversing" mean, as a category? A **Reversing** challenge gives you someone else's already-built app (an APK) and asks you to figure out, by reading its code, some secret value it's hiding -- a PIN, a password, a flag string.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**).

Why is there no exploit app to build here? Reversing challenges usually aren't solved by writing an Android app at all -- they're solved by reading the decompiled logic and then writing a small script (usually Python) that re-implements or defeats that same logic on a regular computer, offline.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/03 Goingnative\Original Target APK.apk
```

How the solution was built

No Android app was built for this one at all. It was solved by **decompiling the target with jadx** and then writing a small **Python script** (or, in a couple of cases, working the logic out by hand) that re-implements or defeats the exact same check on a regular computer, offline. The recovered value was then typed directly into the challenge's own flag box.

The vulnerability / technique, in plain terms

Parsed the target's ELF section headers and dynamic symbol table directly (no objdump/readelf available) to locate the exported JNI function `Java_com_mobisec_gonative_FlagChecker_helloFromTheOtherSide`. Disassembled it with Capstone (x86_64), resolving every RIP-relative LEA instruction's actual target address, and cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncpy/strncmp/strcmp rather than guessing from instruction patterns alone. This recovered the exact 12-byte string ("native_is_so") and integer (31337 / 0x7a69) the native code requires, cross-validated against the Java-side FC.checkFlag() bytecode logic via a from-scratch Dalvik disassembler with a complete, correctly-widthed opcode table.

Files to open

1. Open the target APK in jadx-gui:

   `C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/03 Goingnative\Original Target APK.apk`

2. FlagChecker.java (jadx decompiled):

   `C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/03 Goingnative\Analysis and Scripts\Reversing/`

In jadx-gui's left tree: Source code -> com -> mobisec -> gonative -> FlagChecker.java

3. disasm.py:

   `C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/03 Goingnative\Analysis and Scripts\Reversing/`

What to say -- explanation of the exploit, step by step

1. I extracted the target's native .so library and parsed its ELF section headers and dynamic symbol table directly, since objdump/readelf weren't available.

2. I located the exported JNI function and disassembled it with Capstone, resolving every RIP-relative instruction's actual target address.

3. I cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncmp/strcmp rather than guessing from instruction patterns alone.

4. I recovered the exact string ("native_is_so") and integer (31337) the native code requires, and cross-validated this against the Java-side bytecode logic with a from-scratch Dalvik disassembler.

5. I submitted the recovered values and the platform confirmed the solve.

If an emulator isn't available

If asked how you got/confirmed the flag: "I decompiled the target with jadx, recovered the secret value with a script (no exploit app was built or uploaded for this one), and typed the recovered flag directly into the challenge site's own flag box." There is no APK submission or platform analysis log for this challenge -- don't claim one.

Result

The flag this challenge produced:

`MOBISEC{native_is_so} 31337 (validator input; confirmed SOLVED on live platform 2026-09-24)`

Source: Validator input rigorously confirmed via disassembly; solve timestamp confirmed live from /user